LAB REPORT 3: CPU SCHEDULING ALGORITHMS (FCFS)

Course: Operating Systems Lab

Student Name: Aryan Nautiyal
Roll Number: 251210015

Date: 2026-08-11
Compiler: `clang++ -O3`

1.1 Aim

To simulate the Non-preemptive First-Come, First-Served (FCFS) CPU scheduling algorithm, model CPU allocation based on process arrival times, print a Gantt chart representation, and compute critical performance metrics

1.2 Theory

FCFS is a non-preemptive CPU scheduling strategy managed by a queue. When a process enters the ready queue, its Process Control Block (PCB) is linked to the tail of the queue. The CPU is allocated to the process at the head of the queue.

Definitions

- **Arrival Time (AT):** Time at which a process enters the ready queue.
- **Burst Time (BT):** Execution time required by the process on the CPU.
- **Start Time (ST):** Time at which the process gets CPU allocation for the first time.

$$\mathrm{ST}_i = \max(\mathrm{CT}_{i-1}, \mathrm{AT}_i)$$

- **Completion Time (CT):** Time at which process finishes execution.

$$\mathrm{CT}_i = \mathrm{ST}_i + \mathrm{BT}_i$$

- **Turnaround Time (TAT):** Total time spent between process arrival and completion.

$$\mathrm{TAT} = \mathrm{CT} - \mathrm{AT}$$

- **Waiting Time (WT):** Total time spent waiting in the ready queue.

$$\mathrm{WT} = \mathrm{TAT} - \mathrm{BT}$$

- **Response Time (RT):** Time from process arrival to its first execution turn. For non-preemptive FCFS, $\mathrm{RT} = \mathrm{ST} - \mathrm{AT} = \mathrm{WT}$.
- **CPU Utilization:** Percentage of total schedule time the CPU was actively processing.

$$\text{CPU Utilization} = \left(\sum \frac{\mathrm{BT}_i}{\text{Total Schedule Time}} \right) \times 100\%$$

- **Throughput:** Number of completed processes per unit time.

$$\text{Throughput} = \frac{N}{\text{Total Schedule Time}}$$

Only weakness is **Convoy effect**, when a resource intensive process blocks smaller processes from utilizing the CPU.

1.3 C++ Source Code

```cpp
include <algorithm>
#include <iomanip>
#include <iostream>
#include <vector>

struct Process {
  int pid;
  int arrival_time;
  int burst_time;
  int start_time;
  int completion_time;
  int turnaround_time;
  int waiting_time;
  int response_time;
};
```

```cpp
void runFCFS(std::vector<Process> &processes) {
  // Sort processes based on Arrival Time, then PID
  std::sort(processes.begin(), processes.end(),
            [](const Process &a, const Process &b) {
              if (a.arrival_time != b.arrival_time)
                return a.arrival_time < b.arrival_time;
              return a.pid < b.pid;
            });

  int current_time = 0;
  int total_idle_time = 0;

  for (auto &p : processes) {
    if (current_time < p.arrival_time) {
      total_idle_time += (p.arrival_time - current_time);
      current_time = p.arrival_time;
    }

    p.start_time = current_time;
    p.completion_time = current_time + p.burst_time;
    p.turnaround_time = p.completion_time - p.arrival_time;
    p.waiting_time = p.turnaround_time - p.burst_time;
    p.response_time = p.start_time - p.arrival_time;

    current_time = p.completion_time;
  }

  std::cout << "\n================================================================"
               "==========\n";
  std::cout << std::setw(6) << "PID" << std::setw(8) << "AT" << std::setw(8)
            << "BT" << std::setw(8) << "ST" << std::setw(8) << "CT"
            << std::setw(8) << "TAT" << std::setw(8) << "WT" << std::setw(8)
            << "RT" << "\n";
  std::cout << "----------------------------------------------------------------"
               "--------\n";

  double total_tat = 0, total_wt = 0, total_bt = 0;
  for (const auto &p : processes) {
    total_tat += p.turnaround_time;
    total_wt += p.waiting_time;
    total_bt += p.burst_time;

    std::cout << std::setw(6) << ("P" + std::to_string(p.pid)) << std::setw(8)
              << p.arrival_time << std::setw(8) << p.burst_time << std::setw(8)
              << p.start_time << std::setw(8) << p.completion_time
              << std::setw(8) << p.turnaround_time << std::setw(8)
              << p.waiting_time << std::setw(8) << p.response_time << "\n";
  }
  std::cout << "================================================================"
               "========\n";
  double cpu_utilization = (total_bt / current_time) * 100.0;
  double throughput = static_cast<double>(processes.size()) / current_time;

  std::cout << std::fixed << std::setprecision(2);
  std::cout << "\n--- Performance Metrics ---\n";
  std::cout << "Average Turnaround Time : " << total_tat / processes.size()
            << " units\n";
  std::cout << "Average Waiting Time    : " << total_wt / processes.size()
            << " units\n";
  std::cout << "CPU Utilization         : " << cpu_utilization << " %\n";
  std::cout << "Throughput              : " << throughput
            << " processes/unit time\n";
```

```
}

int main() {
  std::vector<Process> processes = {{1, 0, 2, 0, 0, 0, 0, 0},
                                    {2, 2, 3, 0, 0, 0, 0, 0},
                                    {3, 4, 5, 0, 0, 0, 0, 0},
                                    {4, 6, 2, 0, 0, 0, 0, 0}};

  runFCFS(processes);
  return 0;
}
```

1.4 Output

```
~$ ./fcfs

========================================================================
   PID      AT      BT      ST      CT     TAT      WT      RT
------------------------------------------------------------------------
    P1       0       2       0       2       2       0       0
    P2       2       3       2       5       3       0       0
    P3       4       5       5      10       6       1       1
    P4       6       2      10      12       6       4       4
========================================================================

--- Performance Metrics ---
Average Turnaround Time : 4.25 units
Average Waiting Time    : 1.25 units
CPU Utilization         : 100.00 %
Throughput              : 0.33 processes/unit time
```

1.5 Gantt Chart

P1	P2	P3	P4

0 2 5 10 12